DEFINE ANALYZER

Copy page

> **Note**
>
> Before SurrealDB version 3.0.0, the `FULLTEXT ANALYZER` clause used the syntax `SEARCH ANALYZER`.

In the context of a database, an analyzer processes text so that it can be searched. It is defined by its name, a set of tokenizers, and a collection of filters.

The output of an analyzer can be experimented with by using the `search::analyze()` function.

Requirements

- You must be authenticated as a root, namespace, or database user before you can use the `DEFINE ANALYZER` statement.
- You must select your namespace and database before you can use the `DEFINE ANALYZER` statement.

Statement syntax

SurrealQL Syntax | Railroad Diagram

```
SurrealQL Syntax

DEFINE ANALYZER [ OVERWRITE | IF NOT EXISTS ] @name [ FUNCTION
  @function ] [ TOKENIZERS @tokenizers ] [ FILTERS @filters ] [
  COMMENT @string ]
```

Skip to content

The `FUNCTION` clause

The `FUNCTION` clause runs a preprocessing step on the initial input before tokenizers and filters run. The reference must be a **function path** (not a call - omit parentheses), and the function must take and return a `string`.

You can use either:

- a `fn::` user-defined function defined with `DEFINE FUNCTION`, or
- a `mod::` function from a Surrealism extension module defined with `DEFINE MODULE` (requires the `surrealism` experimental capability).

```
DEFINE FUNCTION fn::backwardsify($input: string) -> string {
    $input.split('').fold('', |$a, $b| $b + $a);
};

DEFINE ANALYZER backwards FUNCTION fn::backwardsify TOKENIZERS blank;

search::analyze("backwards", "I like SurrealDB");
```

A Surrealism module function works the same way once the module is registered:

```
DEFINE ANALYZER custom FUNCTION mod::demo::alter_string TOKENIZERS cla
```

Output

```
[
    'BDlaerruS',
    'ekil',
    'I'
]
```

Tokenizers

Tokenizers are responsible for breaking down a given text into individual tokens based on a set of instructions. There are a couple of tokenizers that can be used while defining an analyzer as seen below:

Skip to content

The blank tokenizer breaks down a text into tokens by creating a new token each time it encounters a space, tab, or newline character. It's a straightforward way to split text into words or chunks based on whitespace.

```
DEFINE ANALYZER example_blank TOKENIZERS blank;
search::analyze("example_blank", "hello world");
```

Output

```
[
    'hello',
    'world'
]
```

`camel`

The camel tokenizer is used for identifying and creating tokens when the next character in the text is uppercase. This is particularly useful for processing camelCase or PascalCase text, common in programming, to split them into meaningful words.

```
DEFINE ANALYZER example_camel TOKENIZERS camel;
search::analyze("example_camel", "helloWorld");
```

Output

```
[
    'hello',
    'World'
]
```

`class`

The class tokenizer segments text into tokens by detecting changes (digit, letter, punctuation, blank) in the Unicode class of characters. It creates a new token when the character class changes, distinguishing between digits, letters, punctuation, and blanks. This allows for flexible tokenization based on character types.

Skip to content

```
YZER example_class TOKENIZERS class;
```

```
search::analyze("example_class", "123abc!XYZ");
```

Output

```
[
    '123',
    'abc',
    '!',
    'XYZ'
]
```

`punct`

The punct tokenizer generates tokens by breaking the text whenever a punctuation character is encountered. It's suitable for tokenizing sentences or breaking text into smaller units based on punctuation marks.

```
DEFINE ANALYZER example_punct TOKENIZERS punct;
search::analyze("example_punct", "Hello, World!");
```

Output

```
[
    'Hello',
    ',',
    'World',
    '!'
]
```

`segment(language)`

AVAILABLE SINCE: V3.3.0

The segment tokenizer splits text into words using a morphological dictionary, for languages the tokenizers above cannot split correctly.

Those tokenizers decide every boundary from the characters around it, which works for languages that separate words with spaces or case changes. Chinese and Japanese write a [illegible]th no separator at all, so `blank`, `class`, `camel` and `punct` all reduce one to [illegible]orean does use spaces, but attaches grammatical particles to the word they

Skip to content

follow, so `한국어를` ("Korean" + object marker) stays whole and a search for `한국어` never matches it.

`segment` takes one of `chinese`, `japanese` or `korean`:

```
DEFINE ANALYZER example_korean TOKENIZERS blank,segment(korean);
search::analyze("example_korean", "한국어를 배우고 있습니다");
```

Output

```
[
    '한국어',
    '를',
    '배우',
    '고',
    '있',
    '습니다'
]
```

The noun `한국어` ("Korean") is now its own token, so it matches independently of the particle attached to it.

Korean written with Hanja is handled as well as Hangul, which matters for older newspapers, legal text and academic writing. The Hanja words are in the dictionary, and a Hangul particle attached to a Hanja stem separates from it:

```
DEFINE ANALYZER example_plain TOKENIZERS blank,class;
search::analyze("example_plain", "政府는 昨日 臨時國務會議를 召集하고");
search::analyze("example_korean", "政府는 昨日 臨時國務會議를 召集하고");
```

Output

```
[
    '政府는',
    '昨日',
    '臨時國務會議를',
    '召集하고'
]
```

Skip to content

```
    '政府',
```

```
    '는',
    '昨日',
    '臨時',
    '國務',
    '會議',
    '를',
    '召集',
    '하',
    '고'
]
```

`政府` and `會議` separate from the particles that follow them, and `臨時國務會議` separates into the three words it is built from, so each is searchable on its own.

The dictionary covers modern standard Korean. Archaic Hangul written with the old jamo, including the arae-a (`ㆍ`) still used for Jeju, has no entries and is returned whole:

```
search::analyze("example_korean", "ᄒᆞᄂᆞ");
```

Output

```
[
    'ᄒᆞᄂᆞ'
]
```

Such text is not separated into words, though a boundary between an archaic sequence and a modern syllable is still found, because the two are written with different Unicode blocks:

```
search::analyze("example_korean", "한ᄒᆞᄂᆞ글");
```

Output

```
[
    '한',
    'ᄒᆞᄂᆞ글'
]
```

Skip to content

ce shows how much is at stake. Without `segment`, the space-separated every word joined to the particle that follows it:

```
DEFINE ANALYZER example_spaces TOKENIZERS blank,class;
search::analyze("example_spaces", "서울에서 부산까지 기차를 타고 갑니다");
```

Output

```
[
    '서울에서',
    '부산까지',
    '기차를',
    '타고',
    '갑니다'
]
```

Neither `서울` nor `부산` is a token, so neither city can be searched for. `segment(korean)` separates each noun from its particle:

```
search::analyze("example_korean", "서울에서 부산까지 기차를 타고 갑니다");
```

Output

```
[
    '서울',
    '에서',
    '부산',
    '까지',
    '기차',
    '를',
    '타',
    '고',
    '갑니다'
]
```

Japanese (with the small exception of spaced kana-only text for children or language learners) and Chinese need no other tokenizer, since there are no spaces to split on first:

```
DEFINE ANALYZER example_japanese TOKENIZERS segment(japanese);
lyze("example_japanese", "東京都に住んでいます");
```

Skip to content

Output

```
[
    '東京',
    '都',
    'に',
    '住ん',
    'で',
    'い',
    'ます'
]
```

Longer text separates in the same way, including place names written as compounds:

```
search::analyze("example_japanese", "京都駅から大阪駅まで電車で行きます");
```

Output

```
[
    '京都',
    '駅',
    'から',
    '大阪',
    '駅',
    'まで',
    '電車',
    'で',
    '行き',
    'ます'
]
```

`京都駅` separates into the city and the station, so a search for `京都` reaches this text.

One dictionary is used per language, but the search over it is the same for all three, so Japanese and Korean separate along the same joins. The sentence above and its Korean translation come apart morpheme for morpheme:

```
DEFINE ANALYZER example_korean TOKENIZERS blank,segment(korean);
    lyze("example_korean", "도쿄도에 살고 있습니다");
```

Skip to content

Output

```
[
    '도쿄',
    '도',
    '에',
    '살',
    '고',
    '있',
    '습니다'
]
```

Japanese	Korean	
東京	도쿄	Tokyo
都	도	metropolis
に	에	location
住ん	살	live
で	고	joins the two verbs
い	있	continuing state
ます	습니다	polite ending

Both languages attach grammatical endings to a stem, and the segmenter separates each ending into a token of its own. That is why a search for the stem alone finds either sentence.

The Japanese dictionary is modern, so older kana is not separated reliably. The kana iteration mark `ゝ`, which stands for a repeat of the kana before it, has no entry of its own and breaks the word it sits in:

```
search::analyze("example_japanese", "こころ");
search::analyze("example_japanese", "こゝろ");
```

Output

```
[
    'こころ'
]
```

Skip to content

```
    'こ',
```

```
    'ゝ',
    'ろ'
]
```

Both spell the same word. Pre-1946 spellings, the obsolete kana `ゐ` and `ゑ`, and hentaigana vary in the same way: each is returned whole where nothing matches it, and broken into single kana where the pieces happen to match something else. The kanji iteration mark `々` is not affected, because words written with it, such as `人々` and `時々`, are entries in their own right.

Simplified | Traditional

```
DEFINE ANALYZER example_chinese TOKENIZERS segment(chinese);
search::analyze("example_chinese", "我喜欢数据库");
```

Output

```
[
    '我',
    '喜欢',
    '数据库'
]
```

The dictionary holds both writings, so either segments correctly on its own.

Japanese text read with `segment(chinese)` partly works, which can be misleading. Characters Japanese shares with Chinese are found, and Japanese-only forms fall out as single characters because no dictionary entry contains them. Chinese writes ice as `冰` in both Simplified and Traditional, where Japanese writes `氷`:

```
search::analyze("example_chinese", "冰水");
search::analyze("example_chinese", "氷水");
```

Output

```
[
    '冰水'
```

Skip to content

```
[
    '氷',
    '水'
]
```

Both mean iced water, and `氷水` is a single word to `segment(japanese)`.

Over a longer phrase the effect is uneven, which is what makes it hard to notice. Here the words Chinese shares are recovered and the ones holding a Japanese-only character are not:

```
search::analyze("example_japanese", "北海道氷河時代地図");
search::analyze("example_chinese", "北海道氷河時代地図");
```

Output

```
[
    '北海道',
    '氷河',
    '時代',
    '地図'
]

[
    '北海道',
    '氷',
    '河',
    '時代',
    '地',
    '図'
]
```

`北海道` and `時代` are written the same way in both languages and survive. `氷河` and `地図` do not, because Chinese writes those characters as `冰` and `圖`, so neither compound is in the dictionary and each falls apart. Half the phrase is still searchable, which is why the wrong dictionary can pass a quick check. Use the dictionary for the language the text is written in.

Chinese has no counterpart to that hiragana exception, and the reason is the reverse of what it suggests. Japanese marks its word boundaries by changing script: kanji write the content ana the grammar around them, so the switch from one to the other often ord ends. Text written only in hiragana loses that and has to be spaced instead. Chinese never had it, because everything is written in one script, so there is nothing

Skip to content

to take away. A Chinese reader separates the words by knowing them, which is the same thing the dictionary does here. Text for learners therefore annotates the characters rather than spacing them: zhuyin in Taiwan, pinyin in mainland China. Pinyin is written word by word, so `blank` already separates it and `segment` has nothing to add. Where both are available, the two agree on ordinary prose: `我在图书馆学习数据库设计` segments into the same six words that pinyin writes as `Wǒ zài túshūguǎn xuéxí shùjùkù shèjì`.

They part company on the names of things. Pinyin writes the parts of a proper name separately, while the dictionary holds the whole name as one entry:

	segment(chinese)	pinyin
`北京大学`	`北京大学`	`Běijīng Dàxué`
`中华人民共和国`	`中华人民共和国`	`Zhōnghuá Rénmín Gònghéguó`

A search for `北京` therefore does not reach a document containing `北京大学`. Index the shorter name as well where both should match. Zhuyin is not in the dictionary, so a run of it is returned whole:

```
search::analyze("example_chinese", "ㄋㄧˇㄏㄠˇ");
```

Output

```
[
    'ㄋㄧˇㄏㄠˇ'
]
```

Text stored as zhuyin is therefore not searchable a word at a time. Spacing it does not recover the words either, because zhuyin is written one group per syllable while a word may be several syllables long. `数据库` is one word of three:

```
search::analyze("example_chinese", "ㄕㄨˋ ㄐㄩˋ ㄎㄨˋ");
```

Output

```
[
    'ㄕㄨˋ',
    'ㄐㄩˋ',
    'ㄎㄨˋ'
```

Skip to content

Word boundaries are decided by the dictionary rather than by character count, so words of one, two and three characters separate from each other in the same sentence:

Simplified | Traditional

```
search::analyze("example_chinese", "我在图书馆学习数据库设计");
```

Output

```
[
    '我',
    '在',
    '图书馆',
    '学习',
    '数据库',
    '设计'
]
```

An analyzer may declare at most one `segment`. It runs after the other tokenizers, splitting the pieces they produced, so combining it with `blank` or `class` is useful for text that mixes scripts.

Dictionaries

Each language is segmented with its own dictionary. A dictionary holds the words of one language rather than the characters of one script, which matters because Han characters are used to write several languages. `segment(chinese)` knows Chinese words, not Han characters in general, so text in another language that borrows the script is read as though it were Chinese.

Historical Vietnamese, written in Chữ Nôm, shows what that produces. Characters borrowed from Chinese are found, because they are Chinese words, while characters invented for Vietnamese have no entries and run together:

```
search::analyze("example_chinese", "越南");
search::analyze("example_chinese", "艾獸");
```

Output

Skip to content

```
[
```

```
    '越南'
]

[
    '爻獸'
]
```

The first is found as the Chinese word for Vietnam. The second is two Vietnamese words, `một người` ("one person"), returned as a single token. Spacing would not recover them either: Vietnamese separates syllables rather than words, so `blank` would give syllables in the same way it does for pinyin.

A dictionary covers the character forms it was built from. The Chinese dictionary holds both Simplified and Traditional writings, so each segments correctly, but it treats them as separate words rather than folding one into the other:

```
search::analyze("example_chinese", "一样");
search::analyze("example_chinese", "一樣");
```

Output

```
[
    '一样'
]

[
    '一樣'
]
```

Both are the same word, and both produce a single token, but the tokens differ. Mixing the two writings in one piece of text is safe as long as each word is written consistently, because every word is looked up in the form it appears in. A word that mixes them internally matches nothing and falls back to single characters:

```
search::analyze("example_chinese", "数據库");
```

Output

Skip to content

```
    '数',
```

```
    '據',
    '库'
]
```

`数据库` and `數據庫` are both in the dictionary; the half-converted `数據库` is not. This matters where text has been through an unreliable converter, because the result still reads correctly and only the tokens show the problem. A document indexed in Traditional characters is therefore not matched by a Simplified query, or the reverse. Where a collection mixes the two, convert to one form before indexing and convert queries the same way. The same applies to Japanese text written with older character forms.

The released binaries carry all three, so `segment` works with nothing on disk and no configuration. That covers the downloads from install.surrealdb.com, the Homebrew formula and the official Docker images, which package those same binaries. The dictionaries are the reason those downloads are substantially larger than they would otherwise be.

Replacing them from disk is for the cases the shipped dictionaries do not cover: a custom or updated dictionary, or a build made without them. Set `SURREAL_SEGMENT_DICTIONARY_PATH` to a directory holding one subdirectory per language. The names are fixed, and each is the language it serves rather than the dictionary behind it: `korean`, `japanese`, `chinese`. Only the languages you use need to be present:

Bash PowerShell

```
# /opt/surreal/dicts needs a subdirectory only for the languages it re
# the rest keep using the dictionaries built into the binary
SURREAL_SEGMENT_DICTIONARY_PATH="/opt/surreal/dicts" \
  surreal db start --user root --pass secret
```

The directory overrides the built-in dictionary for each language it carries, and leaves the rest alone. A directory holding only `korean` therefore replaces Korean while Japanese and Chinese keep using the dictionaries in the binary.

Only an absent dictionary falls back that way. A dictionary that is present but cannot be read, and a directory named by `SURREAL_SEGMENT_DICTIONARY_PATH` that is not there at all, are errors rather than a silent fall back to the built-in one, so a misconfigured path is reported instead of quietly segmenting with something other than what you named.

Not every build embeds the dictionaries. Builds from source do not unless the `cjk` feature is

Skip to content

ther does the WebAssembly package, which could not carry them. If the

dictionary for a language can be found neither way, the `DEFINE ANALYZER` statement naming it fails, rather than being accepted and silently indexing unsegmented text.

Filters

Filters take on the task of transforming these tokens for further processing and analysis.

`ascii`

The ascii filter is responsible for processing tokens by replacing or removing diacritical marks (accents and special characters) from the text. It helps standardize text by converting accented characters to their basic ASCII equivalents, making it more suitable for various text analysis tasks.

```
DEFINE ANALYZER example_ascii TOKENIZERS class FILTERS ascii;
search::analyze("example_ascii", "résumé café");
```

Output

```
[
    'resume',
    'cafe'
]
```

`lowercase`

The lowercase filter converts tokens to lowercase, ensuring that text is consistently in lowercase format. This is often used to make text case-insensitive for search and analysis purposes.

```
DEFINE ANALYZER example_lowercase TOKENIZERS class FILTERS lowercase;
search::analyze("example_lowercase", "Hello World");
```

Output

```
[
    'hello',
    'world'
```

Skip to content

`uppercase`

The uppercase filter converts tokens to uppercase, ensuring text consistency in uppercase format. It can be useful when case-insensitivity is required for specific analysis or search operations.

For example, if you had the text "**Hello World**", the uppercase filter would create two tokens, "**HELLO**", "**WORLD**". Below is an example of how to use the uppercase filter:

```
DEFINE ANALYZER example_uppercase TOKENIZERS class FILTERS uppercase;
search::analyze("example_uppercase", "Hello World");
```

Output

```
[
    'HELLO',
    'WORLD'
]
```

`edgengram(min,max)`

The edgengram filter is used to create tokens that represent prefixes of terms. It generates a sequence of tokens that gradually build up a term, which can be useful for autocomplete or searching based on partial words. It accepts two parameters `min` and `max` which define the minimum and maximum amount of characters in the prefix.

For example, if you had the text "**apple banana**", the edgengram filter would create six tokens, "**a**", "**ap**", "**app**", "**b**", "**ba**", "**ban**". Below is an example of how to use the edgengram filter:

```
DEFINE ANALYZER example_edgengram TOKENIZERS class FILTERS
  edgengram(1,3);
search::analyze("example_edgengram", "apple banana");
```

```
[
    'a',
    'ap',
    'app',
    'b',
    'ba',
    'ban'
```

Skip to content

```
]
```

`mapper(path)`

The mapping filter is designed to enable lemmatization within SurrealDB.

Lemmatization is the process of reducing words to their base or dictionary form. The mapper mechanism allows users to specify a custom dictionary file that maps terms to their base forms. This dictionary file is then used by SurrealDB's analyzer to standardize terms as they are indexed, improving search consistency.

This is particularly useful for handling irregular verbs and other terms that the default "snowball" filter cannot handle. Lemmatization files are easy to put together and to find online, making it possible to customise full-text search for smaller languages.

Filesystem allowlist

A `DEFINE ANALYZER` statement with `mapper('<path>')` opens the dictionary file on the **host filesystem** when the analyzer is defined. Access is gated by `SURREAL_FILE_ALLOWLIST`, without which no paths are permitted. Set one or more directories before using `mapper()`:

Bash PowerShell

```
# Colon-separated directories
SURREAL_FILE_ALLOWLIST="/var/surreal/dicts:/opt/wordlists" surreal db
```

The path in `mapper()` must resolve to a file **under** an allowed directory. Paths outside the allowlist are rejected at `DEFINE ANALYZER` time.

> **Note**
>
> This allowlist is for analyzer dictionary files only. The experimental files feature uses `SURREAL_BUCKET_FOLDER_ALLOWLIST` instead.

How does the mapper work?

Configuration: In the SQL statement below, the mapper parameter is specified within the [illegible]on. This parameter points to the file that contains the term mappings for lemmatization.

Skip to content

```
DEFINE ANALYZER lemme_english TOKENIZERS blank,class FILTERS
  lowercase,mapper( '../tests/data/lemmatization-en.txt' );

RETURN [
    search::analyze("lemme_english", "He drove and swam"),
];
```

Output

```
[
    [
        'he',
        'drive',
        'and',
        'swim'
    ]
]
```

Dictionary File Structure: The file specified in the mapper parameter must follow this format:

- Each line contains a pair of terms separated by a tab.
- The first term represents the canonical (base form) of the word.
- The second term is the form to be mapped to this base form.

Example file format:

```
drive   driven
drive   drives
drive   driving
drive   drove
swim    swam
swim    swimming
swim    swims
swim    swum
```

Usage: When this analyzer is applied to a text, any word that matches the mapped term in the dictionary file will be replaced by its base form before indexing. This helps ensure consistency by consolidating different forms of a word to a single, standardized entry.

Skip to content

By using this custom dictionary-based mapper, you can control how irregular forms and other variations of terms are indexed, making search behaviour more predictable and comprehensive.

The following example shows how lemmatization can be used to generate a list of words and their respective frequencies. Other notable functionalities in the example are the `string::is_alpha()` function inside `array::filter()` to remove all non-alphabetic strings, the `type::record()` function to construct a record ID from two strings, and an `UPSERT` statement to create a record if one does not exist, or update it otherwise.

```
DEFINE ANALYZER lemme_english TOKENIZERS blank,class FILTERS
  lowercase,mapper( '../tests/data/lemmatization-en.txt' );

LET $text = "The Wheel of Time turns,
  and Ages come and pass,
  leaving memories that become legend. Legend fades to myth,
  and even myth is long forgotten when the Age that gave it birth come
  called the Third Age by some,
  an Age yet to come,
  an Age long past,
  a wind rose in the Mountains of Mist. The wind was not the beginning

LET $words = search::analyze("lemme_english", $text)
    .filter(|$c| $c.is_alpha());
FOR $word IN $words {
    UPSERT type::record("word", $word) SET frequency += 1;
};

SELECT * FROM word WHERE frequency >=3 ORDER BY frequency DESC;
```

Output

```
[
    {
        frequency: 8,
        id: word:the
    },
    {
        quency: 6,
        word:age
    },
```

Skip to content

```
    {
        frequency: 4,
        id: word:a
    },
    {
        frequency: 4,
        id: word:be
    },
    {
        frequency: 4,
        id: word:of
    },
    {
        frequency: 3,
        id: word:and
    },
    {
        frequency: 3,
        id: word:come
    },
    {
        frequency: 3,
        id: word:to
    }
]
```

A mapper can also be used for ad-hoc filtering, as long as the file referenced contains two single words separated by a tab. Take the following file for example:

```
NOT_FOUND   File_not_found
NOT_FOUND   Datei_nicht_gefunden
NOT_FOUND   Fichier_non_trouvé
TIMEOUT Timed_out
TIMEOUT Délai_expiré
TIMEOUT Zeitüberschreitung
```

An analyzer that uses a single mapper filter can then use this lemmatizer to unify multilingual [redacted] into a single output.

Skip to content

```
DEFINE ANALYZER error_filter FILTERS mapper('error_filter.txt');

LET $messages =
    ["File not found", "Datei nicht gefunden", "Zeitüberschreitung"]
    .map(|$word| $word.replace(' ', '_'))
    .join(' ');
search::analyze("error_filter", $messages);
```

Output

```
[
    'NOT_FOUND',
    'NOT_FOUND',
    'TIMEOUT'
]
```

Example using the same mapper to search for errors in multiple languages:

```
DEFINE ANALYZER error_filter FILTERS mapper('error_filter.txt');
DEFINE INDEX OVERWRITE errors
  ON TABLE error FIELDS message FULLTEXT ANALYZER error_filter;

FOR $message IN ["File not found",
  "Datei nicht gefunden",
  "Zeitüberschreitung"] {
    CREATE error SET message = $message.replace(' ',
      '_'),
      at = time::now();
};

SELECT * FROM error WHERE message @@ "NOT_FOUND";
```

Output

```
[
    {
        at: d'2024-11-13T03:56:12.039252Z',
        id: error:acbc044syhnx54wzs3n9,
        message: 'File_not_found'
```

Skip to content

```
    },
    {
        at: d'2024-11-13T03:56:12.043643Z',
        id: error:5ifxic9s750x24ts4zof,
        message: 'Datei_nicht_gefunden'
    }
]
```

`ngram(min,max)`

The ngram filter is used to create a sequence of 'n' tokens from a given sample of text or speech. These items can be syllables, letters, words or base pairs according to the application. It accepts two parameters `min` and `max` which indicates that you want to create n-grams starting from min to size of max.

```
DEFINE ANALYZER example_ngram TOKENIZERS class FILTERS ngram(1,3);
search::analyze("example_ngram", "apple banana");
```

Output

```
[
    'a',
    'ap',
    'app',
    'p',
    'pp',
    'ppl',
    'p',
    'pl',
    'ple',
    'l',
    'le',
    'e',
    'b',
    'ba',
    'ban',
    'a',
    'an',
    'n',
```

Skip to content

```
    'na',
    'nan',
    'a',
    'an',
    'ana',
    'n',
    'na',
    'a'
]
```

`snowball(language)`

The snowball filter applies Snowball stemming to tokens, reducing them to their root form and converts the case to lowercase. The following supported languages can be passed as a parameter in snowball: Arabic, Danish, Dutch, English, French, German, Greek, Hungarian, Italian, Norwegian, Portuguese, Romanian, Russian, Spanish, Swedish, Tamil, Turkish.

```
DEFINE ANALYZER english_snowball TOKENIZERS class FILTERS
  snowball(english);
DEFINE ANALYZER german_snowball TOKENIZERS class FILTERS
  snowball(german);

RETURN [
    search::analyze("english_snowball",
      "Looking at some running cats")
    search::analyze("german_snowball",
      "Sollen wir was trinken gehen?")
];
```

Output

```
[
    [
        'look',
        'at',
        'some',
        'run',
        'cat'
    ],
    [
```

Skip to content

```
        'soll',
        'wir',
        'was',
        'trink',
        'geh',
        '?'
    ]
]
```

Using `IF NOT EXISTS` clause

The `IF NOT EXISTS` clause can be used to define an analyzer only if it does not already exist. You should use the `IF NOT EXISTS` clause when defining an analyzer in SurrealDB if you want to ensure that the analyzer is only created if it does not already exist. If the analyzer already exists, the `DEFINE ANALYZER` statement will return an error.

It's particularly useful when you want to safely attempt to define an analyzer without manually checking its existence first.

On the other hand, you should not use the `IF NOT EXISTS` clause when you want to ensure that the analyzer definition is updated regardless of whether it already exists. In such cases, you might prefer using the `OVERWRITE` clause, which allows you to define an analyzer and overwrite an existing one if it already exists, ensuring that the latest version of the analyzer definition is always in use.

```
-- Create an ANALYZER if it does not already exist
DEFINE ANALYZER IF NOT EXISTS example TOKENIZERS blank;
```

Using `OVERWRITE` clause

The `OVERWRITE` clause can be used to create an analyzer and overwrite an existing one if it already exists. You should use the `OVERWRITE` clause when you want to modify an existing analyzer definition. If the analyzer already exists, the `DEFINE ANALYZER` statement will overwrite the existing analyzer definition with the new one.

```
-- Create an ANALYZER and overwrite if it already exists
DEFINE ANALYZER OVERWRITE example TOKENIZERS blank;
```

Skip to content

More examples

Examples on application of analyzers to indexes can be found in the documentation on `DEFINE INDEX` statement

This example creates an analyzer that tokenizes text based on the class of characters and then applies the lowercase filter to the tokens.

```
-- Creates a simple analyzer removing diacritics marks
DEFINE ANALYZER ascii TOKENIZERS class FILTERS lowercase,ascii;
```

This example creates an analyzer specifically designed for processing English texts.

```
-- Creates an analyzer suitable for English text
DEFINE ANALYZER english TOKENIZERS class FILTERS snowball(english);
```

This example creates an analyzer specifically designed for auto-completion tasks.

```
-- Creates an analyzer suitable for auto-completion.
DEFINE ANALYZER autocomplete FILTERS lowercase,edgengram(2,10);
```

This example creates an analyzer specifically designed for source code analysis.

```
-- Creates an analyzer suitable for source code analysis.
DEFINE ANALYZER code TOKENIZERS class,camel FILTERS lowercase,ascii;
```

Removing analyzers

`REMOVE ANALYZER` fails while any full-text index still references the analyzer. Remove or redefine those indexes first, then remove the analyzer. `REMOVE ANALYZER IF EXISTS` does not bypass this check when the analyzer is still in use.

Learn more about this topic

Skip to content

: The outsiders

Full-text search · SurrealDB University

SurrealDB

The context and memory layer for AI agents

Graph, vector, document, and relational in one engine.
Agent Memory that connects and retrieves context wherever your data lives.

© SurrealDB 2026